

Ramas en Git

Trabajo en paralelo — Fusiones — Resolución de conflictos

¿Qué es una rama?

Una rama es una etiqueta que apunta a un commit concreto y avanza automáticamente cada vez que haces un nuevo commit estando en ella.

Sin ramas, cualquier cambio a medias queda mezclado con el código estable. Con ramas, tienes tu propio espacio de trabajo aislado.

Cómo funciona

- La rama por defecto se llama main (o master en repos más antiguos).
- Al crear una rama nueva, Git pone otra etiqueta en el commit donde estás.
- A partir de ahí, cada rama puede avanzar de forma independiente.
- main y la rama nueva parten del mismo punto y evolucionan por su lado sin pisarse.

```
Commit A → Commit B → Commit C (main sigue aquí)
                        ↓
                Commit D → Commit E (rama experimento)
```

¿Qué es HEAD?

HEAD es un puntero especial que indica en qué rama (y commit) estás trabajando ahora mismo.

- Lo verás constantemente en git log, git status y git lga.
- HEAD -> main significa: "Estoy en la rama main y el commit actual es este."
- Cuando cambias de rama con git switch, HEAD se mueve contigo.
- **HEAD no es la rama — es el puntero que dice 'aquí estás tú'.**

Situaciones habituales de HEAD

Qué ves en git lga	Qué significa
HEAD -> main	Estás en main, en el commit más reciente de esa rama
HEAD -> experimento	Estás en la rama experimento
HEAD -> main, experimento	Ambas ramas apuntan al mismo commit (la rama recién se ha creado)

Comandos básicos de ramas

Comando	Qué hace	Cuándo usarlo
<code>git branch</code>	Lista las ramas y marca cuál es la activa (*)	Para ver en cuál estás
<code>git branch <nombre></code>	Crea una nueva rama en el commit actual	Antes de empezar algo nuevo
<code>git switch <nombre></code>	Te mueve a una rama existente	Para cambiar de contexto
<code>git switch -c <nombre></code>	Crea la rama Y te mueve en un solo paso	Atajo para los dos anteriores
<code>git branch -d <nombre></code>	Borra la rama (seguro: solo si está fusionada)	Tras hacer merge
<code>git branch -D <nombre></code>	Borra la rama forzado (aunque no esté fusionada)	Para descartar trabajo intencionalmente

⚠ Con -d (minúscula), Git se niega a borrar si la rama tiene commits que no están en ninguna otra rama — te protege de perder trabajo.

Con -D (mayúscula), borra sin preguntar. Úsala solo cuando estás seguro.

Visualizar el historial: alias git lga

Para entender qué pasa con las ramas necesitas ver el grafo de commits. El comando completo es:

```
git log --graph --oneline --all --decorate
```

Lo más práctico es crear un alias llamado lga. Ejecútalo una sola vez:

```
git config --global alias.lga "log --graph --oneline --all --decorate"
```

El grafo crece de abajo hacia arriba: el commit más reciente aparece primero. Las etiquetas (main, HEAD, nombres de rama) aparecen junto al commit al que apuntan.

Así se ve git lga con dos ramas:

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
$ git status
On branch main
nothing to commit, working tree clean

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
$ git lga
* 79cd2c6 (HEAD -> main) Creando estilos

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
$
```

El ciclo completo — Paso 1: punto de partida

Siempre partimos de main limpio. Comprobamos que no hay cambios pendientes antes de crear la rama.

```
git status  
git lga
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
```

```
$ git status
```

```
On branch main
```

```
nothing to commit, working tree clean
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
```

```
$ git lga
```

```
* 79cd2c6 (HEAD -> main) Creando estilos
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
```

```
$
```

El ciclo completo — Paso 2: crear la rama

git branch crea una nueva etiqueta en el commit actual. HEAD sigue en main — no te has movido.

```
git branch experimento-fondo
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
```

- ```
$ git branch experimento-fondo
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
```

- ```
$ git lga
```

- *

```
79cd2c6 (HEAD -> main, experimento-fondo) Creando estilos
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
```

- ```
$
```

# El ciclo completo — Paso 3: moverse a la rama

git switch mueve HEAD a la nueva rama. Los archivos cambian al estado de esa rama (ahora mismo es el mismo que main).

```
git switch experimento-fondo
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
```

- \$ git checkout experimento-fondo  
Switched to branch 'experimento-fondo'

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (experimento-fondo)
```

- \$ git status  
On branch experimento-fondo  
nothing to commit, working tree clean

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (experimento-fondo)
```

```
○ $ █
```



# El ciclo completo — Paso 4: commits en la rama

La rama activa avanza con cada commit. main se queda intacta — esos cambios no la tocan.

```
git add estilos.css
git commit -m "Fondo cambiado a rojo"
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (experimento-fondo)
$ git add estilos.css
warning: in the working copy of 'estilos.css', LF will be replaced by CRLF the next time Git touches it

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (experimento-fondo)
$ git commit -m "Fondo cambiado a rojo"
[experimento-fondo 77bddb2] Fondo cambiado a rojo
1 file changed, 1 insertion(+)

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (experimento-fondo)
$ git lga
* 77bddb2 (HEAD -> experimento-fondo) Fondo cambiado a rojo
* 79cd2c6 (main) Creando estilos
```

# El ciclo completo — Pasos 5 y 6: volver y fusionar

## 5. Volver a main

```
git switch main
```

Los archivos recuperan el estado de main. Los cambios siguen seguros en la rama.

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (experimento-fondo)
$ git checkout main
Switched to branch 'main'

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
$ cat estilos.css

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
$
```

## 6. Fusionar (Fast-forward)

```
git merge experimento-fondo
```

main no ha avanzado → Git mueve el puntero hacia adelante. No se crea ningún commit extra.

# El ciclo completo — Resultado de la fusión y limpieza

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
• $ git lga
 * 77bddb2 (experimento-fondo) Fondo cambiado a rojo
 * 79cd2c6 (HEAD -> main) Creando estilos

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
• $ git merge experimento-fondo
Updating 79cd2c6..77bddb2
Fast-forward
 estilos.css | 1 +
 1 file changed, 1 insertion(+)

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
• $ git lga
 * 77bddb2 (HEAD -> main, experimento-fondo) Fondo cambiado a rojo
 * 79cd2c6 Creando estilos

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
○ $
```

## 7. Limpiar — borrar la rama

```
git branch -d experimento-fondo
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
• $ git branch -d experimento-fondo
Deleted branch experimento-fondo (was 77bddb2).

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
• $ git lga
 * 77bddb2 (HEAD -> main) Fondo cambiado a rojo
 * 79cd2c6 Creando estilos

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
○ $
```

# Fusiones: Fast-forward

El comando es siempre el mismo: primero te sitúas en la rama destino, luego fusionas:

```
git switch main
git merge nombre-de-la-rama
```

Fast-forward ocurre cuando main NO ha avanzado desde que creaste tu rama. Git simplemente mueve el puntero hacia adelante. El historial queda lineal y limpio — sin commit de fusión.

## ANTES del merge

```
A → B (main)
 \
 C → D (feature, HEAD)
```



## DESPUÉS del merge

```
A → B → C → D
(main, feature, HEAD)
```

# Fusiones: 3-way merge — antes del merge

Ocurre cuando main SÍ ha avanzado mientras trabajabas en tu rama. Las dos líneas han divergido: cada una tiene commits que la otra no tiene.

Así se ve el grafo cuando las dos ramas han divergido:

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
● $ git lga
 * c6e4da0 (funcionalidad-X) Añadiendo botón
 | * ae5ce05 (HEAD -> main, funcionalidad-companyero) Cambio el logo
 | /
 * 77bddb2 Fondo cambiado a rojo
 * 79cd2c6 Creando estilos

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
○ $
```

# Fusiones: 3-way merge — el commit de unión

Git necesita crear un commit de fusión con DOS padres — uno de cada rama. El grafo mostrará el bucle cerrado.

Resultado en el historial tras el merge:

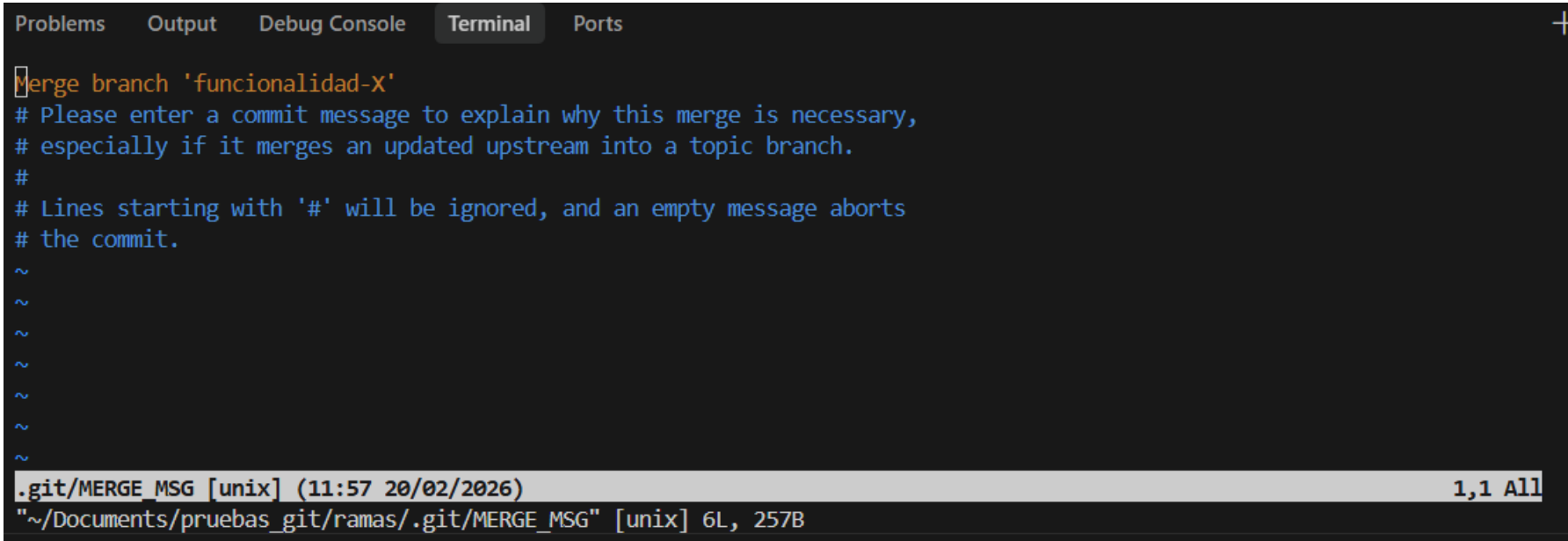
```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
$ git merge funcionalidad-X
Merge made by the 'ort' strategy.
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 index.html

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
$ git lga
* 5d58ef (HEAD -> main) Merge branch 'funcionalidad-X' - cambios nuevos
| \
| * c6e4da0 (funcionalidad-X) Añadiendo botón
| * | ae5ce05 (funcionalidad-companyero) Cambio el logo
| /
|
* 77bddb2 Fondo cambiado a rojo
* 79cd2c6 Creando estilos

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
$
```

# El editor del merge commit

Cuando Git necesita crear un merge commit, abre tu editor de terminal para que confirmes el mensaje:



```
Problems Output Debug Console Terminal Ports
Merge branch 'funcionalidad-X'
Please enter a commit message to explain why this merge is necessary,
especially if it merges an updated upstream into a topic branch.
#
Lines starting with '#' will be ignored, and an empty message aborts
the commit.
~
~
~
~
~
~
~
~
~
.git/MERGE_MSG [unix] (11:57 20/02/2026) 1,1 All
~/Documents/pruebas_git/ramas/.git/MERGE_MSG" [unix] 6L, 257B
```

| Editor                | Guardar y salir                                                  |
|-----------------------|------------------------------------------------------------------|
| Vim (el más habitual) | Pulsa Esc, escribe :wq y pulsa Enter                             |
| Nano                  | Ctrl + O para guardar, Enter para confirmar, Ctrl + X para salir |

# Conflictos: qué son y cuándo ocurren

Un conflicto ocurre cuando Git intenta fusionar dos ramas y encuentra que las mismas líneas de un mismo archivo han sido modificadas de forma diferente en cada una.

Si tú has cambiado la línea 5 de estilos.css en tu rama, y tu compañero también ha cambiado la línea 5 en main, Git no puede elegir por ti. Tendrás que resolver el conflicto a mano.  
Si cada uno ha tocado líneas distintas, Git las combina sin problema.

## Proceso de resolución — siempre el mismo

| Paso | Qué haces                                                        |
|------|------------------------------------------------------------------|
| 1    | Git detiene la operación y te avisa del conflicto                |
| 2    | Abres el archivo — dentro ves las marcas <<<<<< / ===== / >>>>>> |
| 3    | Editas el archivo: borras las marcas y dejas la versión final    |
| 4    | Añades el archivo resuelto: <code>git add archivo.txt</code>     |
| 5    | Cierras el merge: <code>git commit</code>                        |



# Conflictos: las marcas del conflicto

Cuando abres el archivo en conflicto, ves esto:

```
<<<<<< HEAD
Cambios en tu rama actual (main)
=====
Cambios en la rama que entra (nueva-rama)
>>>>>> nueva-rama
```

| Marca             | Qué significa                                       |
|-------------------|-----------------------------------------------------|
| <<<<<< HEAD       | Empieza la versión de la rama donde estás (main)    |
| =====             | Separador entre las dos versiones en conflicto      |
| >>>>>> nueva-rama | Empieza la versión de la rama que intentas fusionar |

Tu tarea: borrar las tres marcas y dejar solo la versión final del texto. Puedes quedarte con una de las dos versiones, o escribir una tercera que combine lo mejor de ambas.

# Conflictos: ejemplo — preparar el escenario

## 1. Crear el archivo y primer commit en main

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
$ vi titulo.txt

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
$ git add .
warning: in the working copy of 'titulo.txt', LF will be replaced by CRLF the next time Git touches it

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
$ git commit -m "Creando fichero titulo"
[main d3f8336] Creando fichero titulo
1 file changed, 1 insertion(+)
create mode 100644 titulo.txt
```

## 2. Crear rama y modificar la misma línea

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (cambio-titulo)
$ vi titulo.txt

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (cambio-titulo)
$ git add .
warning: in the working copy of 'titulo.txt', LF will be replaced by CRLF the next time Git touches it

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (cambio-titulo)
$ git commit -m "Modificando hola universo"
[cambio-titulo 2a4f8eb] Modificando hola universo
1 file changed, 1 insertion(+), 1 deletion(-)

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (cambio-titulo)
```

# Conflictos: ejemplo — cambiar la misma línea en main

Volvemos a main y cambiamos la misma línea. Ahora las dos ramas tienen versiones distintas de esa línea.

## 3. En main, cambiar la misma línea — y ver las ramas divergidas

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
$ vi titulo.txt

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
$ git add .

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
$ git commit -m "Lo cambio por hola planeta"
[main 32f6303] Lo cambio por hola planeta
1 file changed, 1 insertion(+), 1 deletion(-)

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
$ git lga
* 32f6303 (HEAD -> main) Lo cambio por hola planeta
| * 2a4f8eb (cambio-titulo) Modificando hola universo
|/
* d3f8336 Creando fichero titulo
```

# Conflictos: ejemplo — Git detecta el conflicto

Al intentar fusionar, Git para y avisa. El repositorio queda en estado MERGING hasta que resuelvas el conflicto y cierres el merge.

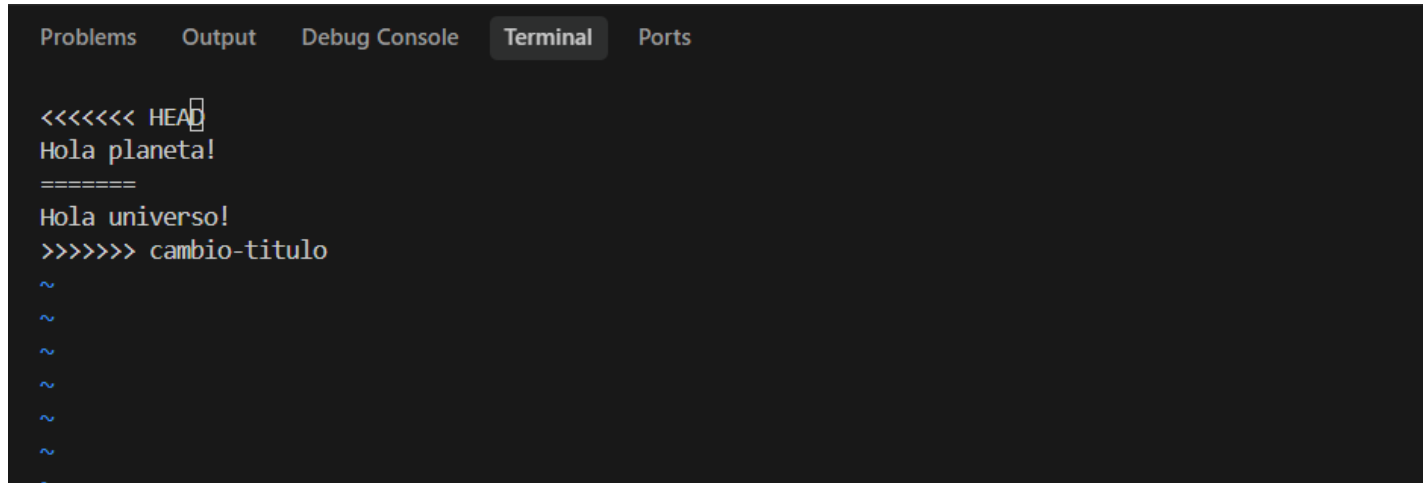
## 4. Intentar fusionar → CONFLICT

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
$ git merge cambio-titulo
Auto-merging titulo.txt
CONFLICT (content): Merge conflict in titulo.txt
Automatic merge failed; fix conflicts and then commit the result.

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main|MERGING)
$
```

# Conflictos: ejemplo — editar y resolver

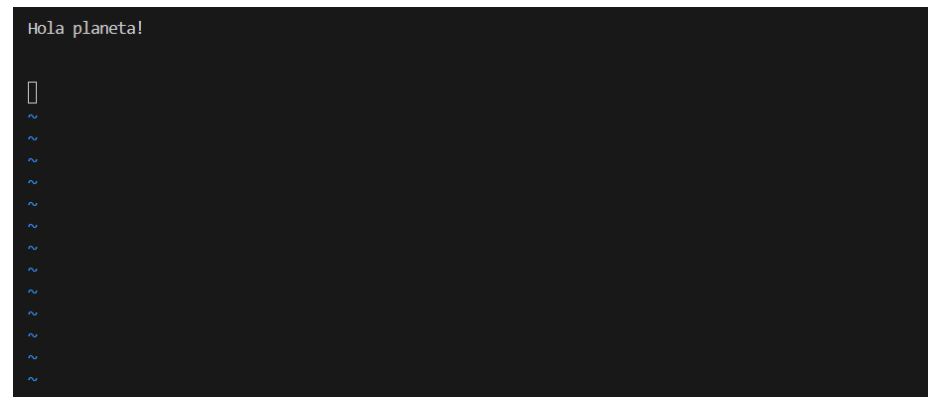
## 5. El archivo con las marcas



```
Problems Output Debug Console Terminal Ports

<<<<<< HEAD
Hola planeta!
=====
Hola universo!
>>>>>> cambio-titulo
~
~
~
~
~
~
~
~
```

## 6. Archivo resuelto — quedas solo con la versión final



```
Hola planeta!
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
```

# Conflictos: ejemplo — cerrar el merge y ver el resultado

## 7. Añadir el archivo resuelto y cerrar el merge

```
git add titulo.txt
git commit -m "Resuelve conflicto en titulo.txt"
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main|MERGING)
• $ vi titulo.txt

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main|MERGING)
• $ git add titulo.txt

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main|MERGING)
• $ git commit -m "Solucionando el conflicto"
[main b300040] Solucionando el conflicto

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
$
```

El historial queda intacto: el commit del conflicto, los commits de cada rama, y el merge commit que los une. Todo queda trazado.

# Conflictos: el historial tras resolverlo

El grafo muestra el merge commit uniendo las dos líneas — el nudo cerrado confirma que el proceso ha terminado:

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
• $ git lga
* b300040 (HEAD -> main) Solucionando el conflicto
| \
| * 2a4f8eb (cambio-titulo) Modificando hola universo
* | 32f6303 Lo cambio por hola planeta
|/
* d3f8336 Creando fichero titulo
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/ramas (main)
• $ cat titulo.txt

Hola planeta!
```

Si hay conflictos en varios archivos a la vez: resuelve uno a uno, haz git add de cada uno que termines, y solo al final haz git commit.

# Ramas desde IntelliJ

No es obligatorio usar la terminal para todo. IntelliJ tiene integración completa con Git.

## Cómo acceder

En la barra inferior de IntelliJ verás el nombre de la rama actual. Haz clic en él para:

- Ver todas las ramas locales y remotas.
- Crear una rama nueva (New Branch).
- Cambiar de rama con un clic (Checkout).
- Fusionar o hacer rebase desde el menú contextual.

Los comandos que hace IntelliJ por debajo son exactamente los mismos que has practicado en terminal. Entenderlos te ayuda a saber qué está pasando cuando algo falla en la interfaz gráfica.



# Ideas clave

## Ramas y HEAD

**git branch <nombre>**

— crea rama sin moverte

**git switch <nombre>**

— cambia de rama

**git switch -c <nombre>**

— crea y cambia en uno

**HEAD**

— apunta siempre a tu rama y commit actual

**git lga**

— grafo visual de todas las ramas

## Fusiones y conflictos

**Fast-forward:**

main no ha avanzado → puntero se mueve, historial lineal

**3-way merge:**

ambas ramas han avanzado → Git crea un merge commit

**Conflicto:**

mismas líneas modificadas → resuelves a mano

**git branch -d**

— borrado seguro (solo si fusionada)

**git branch -D**

— borrado forzado

El flujo básico: `git switch -c mi-rama` → commits → `git switch main` → `git merge mi-rama` → `git branch -d mi-rama`